

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

CO5: Introduction – NumPy Arrays – NumPy Basics- Math – Random – Indexing – Filtering – Statistics – Aggregation – saving data – Data analysis with Pandas –Data Frame – Combining – Indexing – File I/O – Grouping – Filtering – Sorting – Plotting (BL 3)

Name: K. Dedeepya

Reg no: 192324252

2. A financial company provides you with millions of transaction records that must be analysed to detect unusual spending patterns. Due to system limitations, memory and computation time are critical constraints. Describe how you would use NumPy arrays and Pandas Data Frames to perform filtering, grouping, and statistical analysis efficiently. Explain how you would optimize indexing and aggregation operations, and how you would debug performance bottlenecks or incorrect outputs.

Approach:

1. Load the data into a Pandas Data Frame.
2. Convert important numerical columns into suitable NumPy data types.
3. Remove unnecessary columns to reduce memory usage.
4. Use filtering to identify transactions above or below a particular amount.
5. Use grouping to analyse transactions by customer, category, or location.
6. Use aggregation functions such as sum(), mean(), max(), and std() to calculate statistics.
7. Use indexes on frequently searched columns to improve query performance.
8. For millions of records, process the data in smaller chunks instead of loading everything into memory.
9. Check the output using sample records, minimum/maximum values, and summary statistics.
10. Use profiling and timing functions to identify performance bottlenecks.

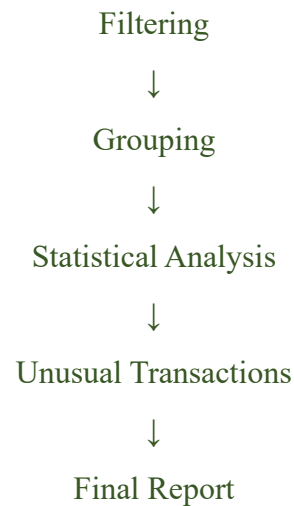
Simple Diagram:

Transaction Data



NumPy / Pandas





Example Code:

```
import pandas as pd

df = pd.read_csv("transactions.csv")

high_value = df[df["Amount"] > 10000]

summary = df.groupby("Customer_ID")["Amount"].agg(["sum", "mean", "max"])

print(summary)
```



```
[2]: import pandas as pd
import numpy as np
# Create a sample DataFrame as an alternative to reading a CSV
data = {
    'Transaction_ID': ['T001', 'T002', 'T003', 'T004', 'T005', 'T006', 'T007', 'T008'],
    'Customer_ID': [101, 102, 101, 103, 102, 104, 101, 103],
    'Amount': [5000, 12000, 7500, 15000, 8000, 3000, 11000, 6000]
}

df = pd.DataFrame(data)
# Filter high-value transactions
high_value = df[df["Amount"] > 10000]
# Group transactions by customer
summary = df.groupby("Customer_ID")["Amount"].agg(["sum", "mean", "max"])
print(summary)
```

Customer_ID	sum	mean	max
101	22500	7833.333333	11000
102	20000	10000.000000	12000
103	21000	10500.000000	15000
104	3000	3000.000000	3000

Conclusion:

NumPy provides fast numerical operations, while Pandas provides efficient Data Frame operations. Filtering, grouping, indexing, aggregation, and chunk processing help analyse millions of transactions efficiently.

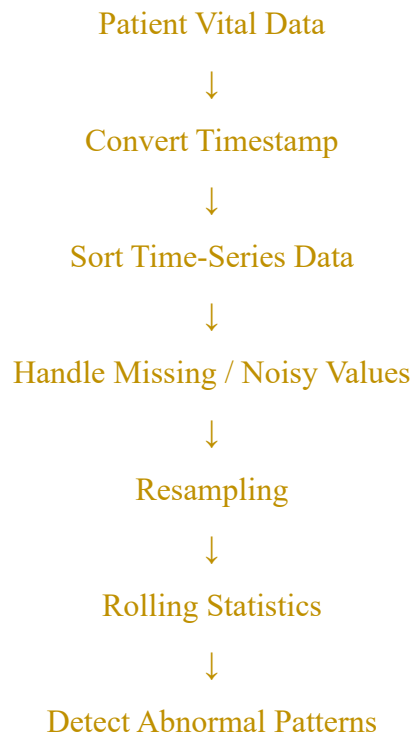
6. A healthcare analytics system requires continuous monitoring of patient vitals stored as time-series data. The dataset includes irregular timestamps and noisy readings. Describe how you would use NumPy and Pandas to clean, resample, and analyse this data. Explain how rolling statistics and aggregation functions can help identify abnormal patterns.

Approach:

1. Load the patient data using Pandas.
2. Convert the timestamp column into a proper datetime format.

3. Sort the data according to timestamp.
4. Remove duplicate or invalid records.
5. Handle missing values using interpolation or forward filling.
6. Remove or identify noisy readings using suitable limits or statistical methods.
7. Set the timestamp as the Data Frame index.
8. Use resampling to convert irregular readings into regular time intervals.
9. Use rolling statistics such as rolling mean to observe changes over time.
10. Use aggregation functions such as mean(), min(), and max() to identify abnormal patterns.

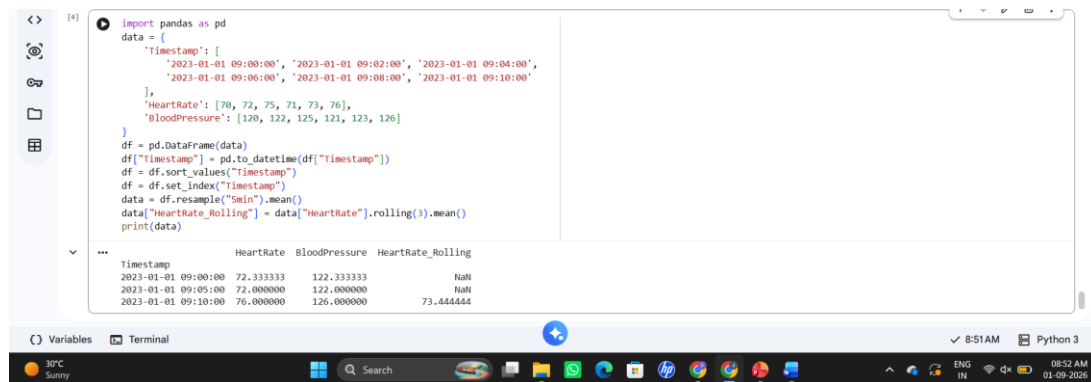
Simple Diagram:



Example Code:

```
import pandas as pd
df = pd.read_csv("vitals.csv")
df["Timestamp"] = pd.to_datetime(df["Timestamp"])
df = df.sort_values("Timestamp")
df = df.set_index("Timestamp")
data = df.resample("5min").mean()
data["HeartRate_Rolling"] = data["HeartRate"].rolling(3).mean()
```

print(data)



```
[4]: import pandas as pd
data = {
    'Timestamp': [
        '2023-01-01 09:00:00', '2023-01-01 09:02:00', '2023-01-01 09:04:00',
        '2023-01-01 09:06:00', '2023-01-01 09:08:00', '2023-01-01 09:10:00'
    ],
    'HeartRate': [70, 72, 75, 71, 73, 76],
    'BloodPressure': [120, 122, 125, 121, 123, 126]
}
df = pd.DataFrame(data)
df['Timestamp'] = pd.to_datetime(df['Timestamp'])
df = df.sort_values('Timestamp')
df = df.set_index('Timestamp')
data = df.resample("5min").mean()
data["HeartRate_Rolling"] = data["HeartRate"].rolling(3).mean()
print(data)
```

Timestamp	HeartRate	BloodPressure	HeartRate_Rolling
2023-01-01 09:00:00	72.333333	122.333333	NaN
2023-01-01 09:05:00	72.000000	122.000000	NaN
2023-01-01 09:10:00	76.000000	126.000000	73.444444

Importance of Rolling Statistics:

A rolling average reduces the effect of sudden noise and helps identify continuous changes in patient vitals. If the rolling value moves outside the normal range, it can indicate a possible abnormal condition.

Conclusion:

Pandas is useful for time-series indexing, sorting, resampling, and rolling calculations. NumPy can be used for fast numerical calculations and statistical operations.

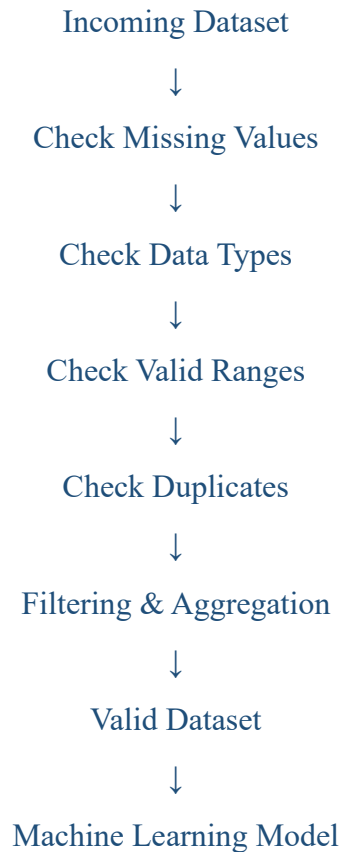
9. You are tasked with building a data validation system for incoming datasets in machine learning pipeline. The system must check for missing values, invalid ranges, and inconsistent formats. Describe how you would implement validation rules using Pandas, and explain how indexing, filtering, and aggregation help enforce data quality constraints.

A machine learning pipeline requires clean and valid input data. Pandas can be used to check missing values, invalid ranges, duplicate records, and inconsistent formats.

Validation Steps:

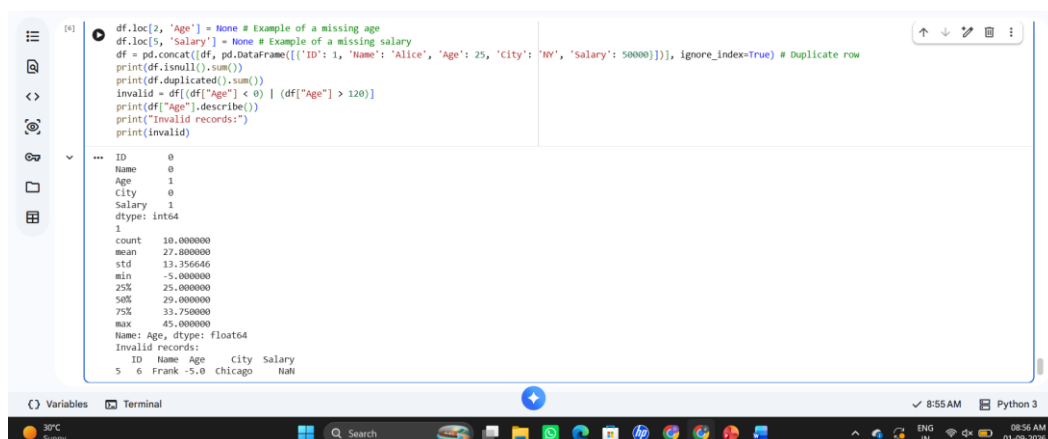
1. Load the incoming dataset using Pandas.
2. Check the number of rows and columns.
3. Check for missing values using `isnull()`.
4. Check for duplicate records using `duplicated()`.
5. Check whether numerical values are within valid ranges.
6. Convert columns into correct data types.
7. Use filtering to find invalid records.
8. Use aggregation to calculate minimum, maximum, average, and other statistics.
9. Use indexing to access important records quickly.
10. Correct or reject invalid records before sending the data to the machine learning model.

Simple Diagram:



Example Code:

```
import pandas as pd
df = pd.read_csv("data.csv")
print(df.isnull().sum())
print(df.duplicated().sum())
invalid = df[(df["Age"] < 0) | (df["Age"] > 120)]
print(df["Age"].describe())
print("Invalid records:")
print(invalid)
```



The screenshot shows a Jupyter Notebook interface with a code cell and its output. The code cell contains the following Python code:

```
[4]: df.loc[2, 'Age'] = None # Example of a missing age
df.loc[5, 'Salary'] = None # Example of a missing salary
df = pd.concat([df, pd.DataFrame([{'ID': 1, 'Name': 'Alice', 'Age': 25, 'City': 'NY', 'Salary': 50000}]), ignore_index=True) # Duplicate row
print(df.isnull().sum())
print(df.duplicated().sum())
invalid = df[(df["Age"] < 0) | (df["Age"] > 120)]
print(df["Age"].describe())
print("Invalid records:")
print(invalid)
```

The output of the code is displayed below the code cell. It shows the sum of null values for each column, the sum of duplicated rows, the description of the 'Age' column, and the invalid records.

```
count    10.000000
mean     27.800000
std      13.356646
min       -5.000000
25%      25.000000
50%      29.000000
75%      33.750000
max       45.000000
Age: Age, dtype: float64
Invalid records:
   ID  Name  Age  City  Salary
5    6  Frank -5.0  Chicago    NaN
```

Role of Indexing, Filtering and Aggregation:

- **Indexing:** Helps access specific records efficiently.
- **Filtering:** Helps identify invalid or unwanted records.
- **Aggregation:** Helps summarize the data and detect unusual values.

Conclusion:

Pandas provides simple and efficient functions for data validation. Validation before model training improves data quality and reduces incorrect machine learning results.

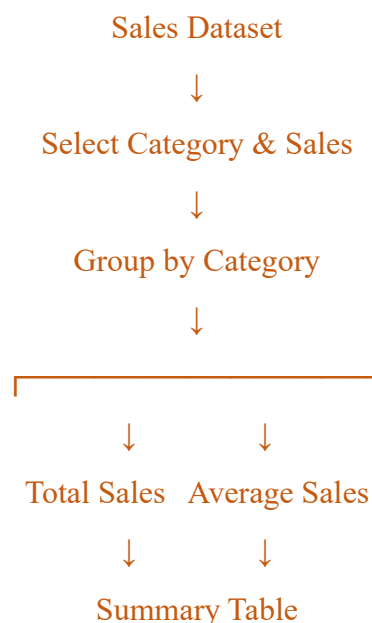
12. A company provides you with sales data, and you are asked to find the total and average sales for each product category. How would you use Pandas grouping and aggregation functions to solve this problem? Explain the steps clearly.

Pandas group by() and aggregation functions can be used to calculate the total and average sales for each product category.

Steps:

1. Load the sales dataset using Pandas.
2. Identify the category and sales columns.
3. Group the data using the product category.
4. Calculate the total sales using sum().
5. Calculate the average sales using mean().
6. Display the result as a summary table.

Simple Diagram:



Example Code:

```
import pandas as pd

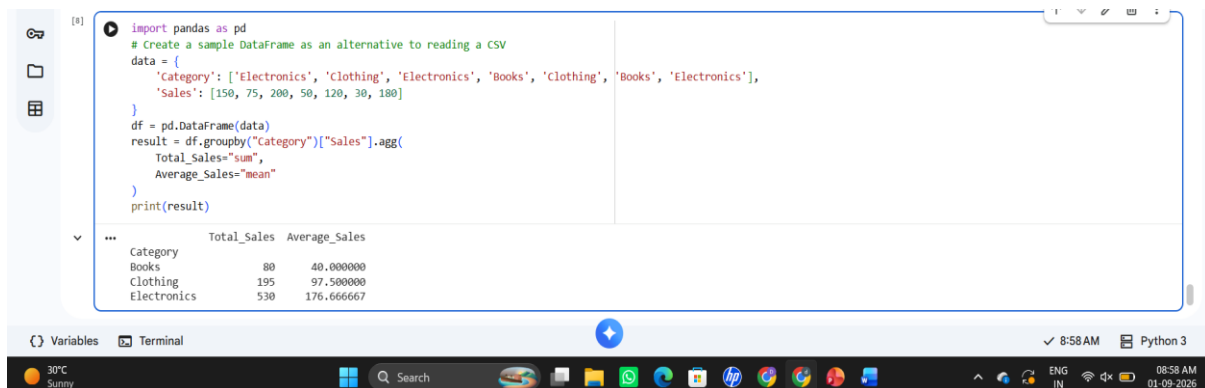
df = pd.read_csv("sales.csv")

result = df.groupby("Category")["Sales"].agg(

    Total_Sales="sum",

    Average_Sales="mean")

print(result)
```



```
[5]: import pandas as pd
# Create a sample DataFrame as an alternative to reading a CSV
data = {
    'Category': ['Electronics', 'Clothing', 'Electronics', 'Books', 'Clothing', 'Books', 'Electronics'],
    'Sales': [150, 75, 200, 50, 120, 30, 180]
}

df = pd.DataFrame(data)
result = df.groupby("Category")["Sales"].agg(
    Total_Sales="sum",
    Average_Sales="mean"
)

print(result)
```

Category	Total_Sales	Average_Sales
Books	80	40.000000
Clothing	195	97.500000
Electronics	530	176.666667

Conclusion:

The groupby() function divides the data into categories, while sum() and mean() calculate the total and average sales. This makes it easy to compare the performance of different product categories.

15. You are given a dataset with both numerical and categorical features, and you need to prepare it for a machine learning model. How would you perform basic preprocessing such as handling missing values, encoding categorical variables, and normalizing numerical features using NumPy and Pandas? Explain the steps briefly.

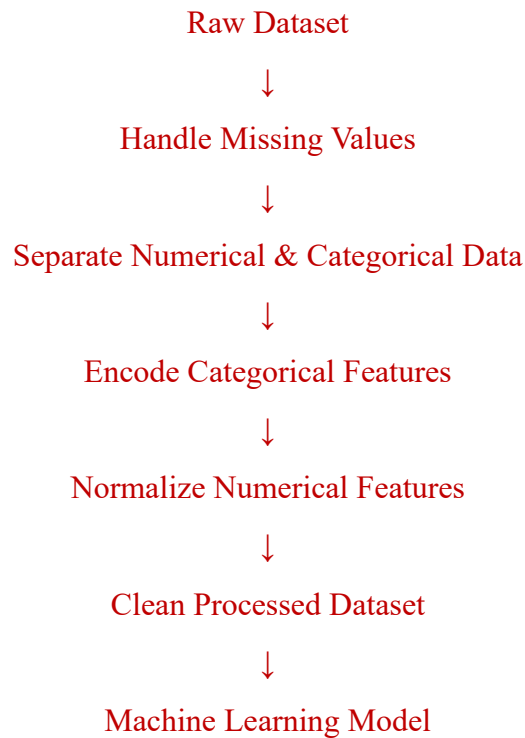
Before using a dataset for machine learning, the data must be cleaned and transformed into a suitable format.

Steps:

1. Load the dataset using Pandas.
2. Identify numerical and categorical columns.
3. Check for missing values.
4. Fill missing numerical values using the mean or median.
5. Fill missing categorical values using the mode.
6. Convert categorical variables into numerical values using encoding.
7. Normalize numerical features so that they have comparable scales.
8. Use NumPy for efficient numerical calculations.
9. Check the final dataset for missing or invalid values.

10. Use the processed dataset as input for the machine learning model.

Simple Diagram:



Example Code:

```
import pandas as pd
import numpy as np

df = pd.read_csv("data.csv")

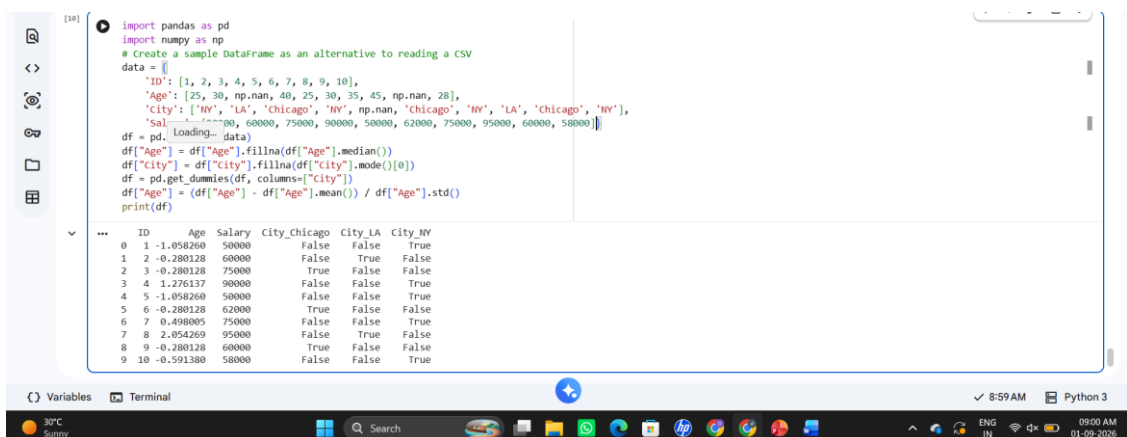
df["Age"] = df["Age"].fillna(df["Age"].median())

df["City"] = df["City"].fillna(df["City"].mode()[0])

df = pd.get_dummies(df, columns=["City"])

df["Age"] = (df["Age"] - df["Age"].mean()) / df["Age"].std()

print(df)
```



```
[10]: import pandas as pd
import numpy as np
# Create a sample DataFrame as an alternative to reading a CSV
data = {
    'ID': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'Age': [25, 30, np.nan, 40, 25, 30, 35, 45, np.nan, 28],
    'City': ['NY', 'LA', 'Chicago', 'NY', np.nan, 'Chicago', 'NY', 'LA', 'Chicago', 'NY'],
    'Sal': [50000, 60000, 75000, 90000, 50000, 62000, 75000, 95000, 60000, 58000]}

df = pd.DataFrame(data)

df["Age"] = df["Age"].fillna(df["Age"].median())
df["City"] = df["City"].fillna(df["City"].mode()[0])
df = pd.get_dummies(df, columns=["City"])
df["Age"] = (df["Age"] - df["Age"].mean()) / df["Age"].std()
print(df)
```

	ID	Age	Salary	City_Chicago	City_LA	City_NY
0	1	-1.058260	50000	False	False	True
1	2	-0.280128	60000	False	True	False
2	3	-0.280128	75000	True	False	False
3	4	1.276137	90000	False	False	True
4	5	-1.058260	50000	False	False	True
5	6	-0.280128	62000	True	False	False
6	7	0.498005	75000	False	False	True
7	8	2.054269	95000	False	True	False
8	9	-0.280128	60000	True	False	False
9	10	-0.591380	58000	False	False	True

Normalization:

Normalization brings numerical values to a similar scale. One common method is:

Normalized Value = (Value - Mean) / Standard Deviation

This prevents features with large numerical values from dominating the machine learning model.

Conclusion:

Pandas is used for data cleaning, missing-value handling, and categorical encoding, while NumPy provides fast numerical operations. Proper preprocessing improves the quality and performance of machine learning models.